



||JAI SRI GURUDEV||
Sri AdichunchanagiriShikshana Trust ®

SJB INSTITUTE OF TECHNOLOGY

(A Constituent Institution of BGS & SJBIT Group of Institution & Hospitals)
No. 67, BGS Health & Education City, Dr. Vishnuvardhan Road, Kengeri, Bengaluru - 560 060



Department of Computer Science & Engineering



Subject: Theory of Computation
Subject Code: 23CSI602

Module 2: Regular Expressions

REGULAR EXPRESSIONS

Language represented in the form of expression (symbols separated by operator) is called regular expression.

$$\Sigma = \{a, b, 0, 1\}$$

Language $L = \Sigma^* = \{a, b, 0, 1\}^* = \{\epsilon, a, b, 0, 1, aa, bb, 00, 11, \dots\}$ Regular

expression $R = \{\epsilon + a + 0 + 1 + aa + bb + 00 + 11, \dots\}$

Before studying regular expression in detail, we will discuss some fundamental rules of the regular expression.

- i. $\Phi + 1 = 1$ (Φ is like number 0).
- ii. $\epsilon + 1 = \epsilon + 1$ (ϵ is like number 1)
- iii. $\phi\epsilon = \phi$
- iv. $\epsilon + \phi = \epsilon$
- v. $1 + 1 = 1$
- vi. $0 + 0 = 0$
- vii. $0 + 1 = 0 + 1$
- viii. $0 + 01 = 0(\epsilon + 1)$
- ix. $0\epsilon = 0$
- x. $0\phi = 0$
- xi. $\Phi^* = \{\epsilon + \phi + \phi\phi \dots\} = \{\epsilon + \phi\} = \epsilon$
- xii. $0^+ + \epsilon = 0^*$
- xiii. $0^* + \epsilon = 0^*$
- xiv. $(1 + \epsilon)^+ = 1^*$
- xv. $(1 + \phi)^+ = 1^+$
- xvi. $00^* = 0^+$
- xvii. $\epsilon + 11^* = \epsilon + 1^+ = 1^*$
- xviii. $(\epsilon + 11)(\epsilon + 11)^* = (\epsilon + 11)^+ = (11)^*$
- xix. $(00)^*(1 + 01) = (00)^*(\epsilon + 0)1 = (\epsilon + 00 + 0000 \dots)(\epsilon + 0)1 = 0^*1$
- xx. Write a regular expression set of strings made up of 0's and 1's ending with 00.
 $RE = (0 + 1)^*00$.
- xxi. Write a regular expression for all the set of string made up of 0's and 1's which begin with 0 and end with 1
 $RE = 0(1 + 0)^*1$
- xxii. Strings begin with 0 or 1 is represented as $(0 + 1)(0 + 1)^*$
- xxiii. All strings begin with 0 or 1 and not having two consecutive 0's
 $RE = (0 + 1)(1 + 10)^*$
- xxiv. Write a regular expression which consists of two consecutive 0's
 $RE = (0 + 1)^*00(0 + 1)^*$
- xxv. All the strings of 0's and 1's whose last two symbols are same
 $RE = (0 + 1)^*(00 + 11)$
- xxvi. Set of all strings which start with either 00 or 11 or end with 00 or 11

- RE=(00+11)(0+1)*+(0+1)*(00+11).
- xxvii. Find the regular expression of the language $L = \{a^n b^m : n \geq 0, m \geq 0\}$
 $L = \{\epsilon + a + aa.. \} \{ \epsilon + b + bb.. \}$
 RE = $a^* b^*$
- xxviii. Find the regular expression of the language $L = \{a^n b^{2m+2} : n \geq 0, m \geq 0\}$
 RE = $(aa)^*(bb)^*bb$.
- xxix. Find the regular expression of the language $L = \{a^{2n} b^m : n \geq 4, m \leq 3\}$
 RE = $(aaaa)^*(\epsilon + b + bb + bbb)$
- xxx. Find the regular expression of the language $L = \{a^n b^m : n + m \text{ is even}\}$
 RE = $(aa)^*(bb)^* + a(aa)^*b(bb)^*$

Conversion from Regular expression to finite automata:

Three basic regular expressions are used in conversion (0+1), 01 and 0^* . The finite automata which accept either 0 or 1 is shown in figure 2.1.

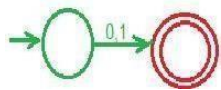


Figure 2.1: Finite automata

But in regular expression each symbol has its own finite automata. Definition

for the finite automata which accept the symbol '1' is

$$M_1 = (Q_1, \Sigma_1, \delta, IS_1, FS_1)$$

$$Q_1 = \{C, D\}; \Sigma_1 = \{1\}; \delta(C, 1) = D; IS = C; FS = D$$

Definition for the finite automata which accept the symbol '0' or '1' is $M =$

$$(Q, \Sigma, \delta, IS, FS)$$

$$Q = \{Q_0, Q_1, U, E, F\} = \{A, B, C, D, E, F\};$$

$$\Sigma = \{0, 1\}$$

δ

$$\delta(A, 0) = B; \delta(C, 1) = D; \delta(E, \epsilon) = \{A, C\}; \delta(D, \epsilon) = F; \delta(D, \epsilon) = F; IS = E; FS = F$$

For the symbol '0' the finite automata is shown in figure 2.2

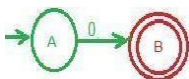


Figure 2.2: Finite automata for regular expression 0.

Definition for the finite automata which accept the symbol '0' is $M_0 =$

$$(Q_0, \Sigma_0, \delta, IS_0, FS_0)$$

$$Q_0 = \{A, B\}; \Sigma_0 = \{0\}; \delta(A, 0) = B; IS = A; FS = B$$

For the symbol '1' is shown in figure 2.3

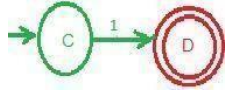


Figure 2.3: Finite automata for regular expression 1.

Definition for the finite automata which accept the symbol '1' is $M_1 =$

$$(Q_1, \Sigma_1, \delta, IS_1, FS_1)$$

$$Q_1 = \{C, D\}; \Sigma_1 = \{1\}; \delta(C, 1) = D; IS = C; FS = D$$

We are joining these two finite automata with disturbing the regular expression $0+1$ as shown in figure 2.4. To join these two finite automata we need more extra states, one for the initial state and second is for the final state, states A and B no more initial states and B and D are no more final states. All these states will now become intermediate states as shown in figure 2.4.

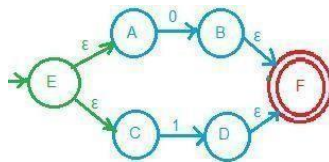


Figure 2.4: Finite automata for regular expression $0+1$.

Definition for the finite automata which accept the symbol '0' or '1' is $M =$

$$(Q, \Sigma, \delta, IS, FS)$$

$$Q = \{Q_0 \cup Q_1 \cup E \cup F\} = \{A, B, C, D, E, F\};$$

$$\Sigma = \{0, 1\}$$

δ

$$\delta(A, 0) = B; \delta(C, 1) = D; \delta(E, \epsilon) = \{A, C\}; \delta(D, \epsilon) = F; \delta(B, \epsilon) = F; IS = E; FS = F$$

For the regular expression 01 the finite automata is shown in figure 2.6.

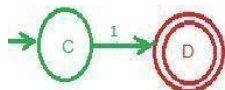


Figure 2.5: Finite automata for regular expression 0 and 1.

Rewrite the expression 01 as $0\epsilon 1$.

We can join the two finite automata shown in figure 2.5 and figure 2.6 without disturbing the regular expression 01 is shown in figure 2.6.

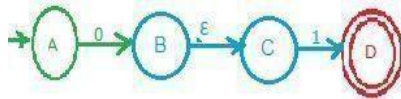


Figure 2.6: Finite automata for regular expression 01.

Definition for the finite automata which accept the string 01 is $M =$

$(Q, \Sigma, \delta, IS, FS)$

$Q = \{Q_0 \cup Q_1\} = \{A, B, C, D\};$

$\Sigma = \{0, 1\}$

δ

$\delta(A, 0) = B; \delta(C, 1) = D; \delta(B, \epsilon) = C$ IS = A; FS = D

Like that we can design the finite automata for 0^* . $0^* =$

$\{\epsilon, 0, 00, 000, \dots\}$

The regular expression for symbol '0' is shown in figure 2.7

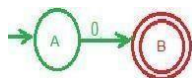


Figure 2.7: Finite automata for regular expression 0.

The designed finite automata accept all the strings of 0^* i.e., $\{\epsilon, 0, 00, 000, \dots\}$

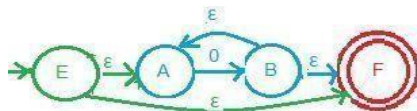


Figure 2.8: Finite automata for regular expression 0^* .

Definition for the finite automata which accept the string 0^* is

$M = (Q, \Sigma, \delta, IS, FS)$

$Q = \{Q_0 \cup E \cup F\} = \{A, B, E, F\};$

$\Sigma = 0$

δ

$\delta(A,0)=B; \delta(E,\epsilon)=\{A,F\}; \delta(B,\epsilon)=\{F,A\}$ IS=E; FS=F

Example 2.1: Design a finite automata for the regular expression 0^*1

- i. Draw the finite automata for the symbol '0'

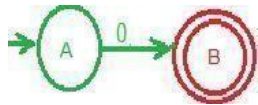


Figure 2.9: Finite automata for regular expression 0.

- ii. Convert the finite automata of symbol '0' to 0^*

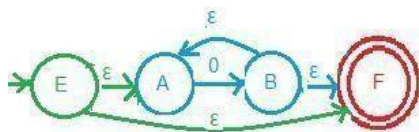


Figure 2.10: Finite automata for regular expression 0^* .

- iii. Draw the finite automata for the symbol '1'

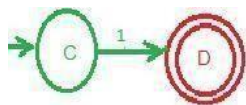


Figure 2.11: Finite automata for regular expression 1.

Join finite automata of 0^* and 1, which will be the solution for the regular expression 0^*1

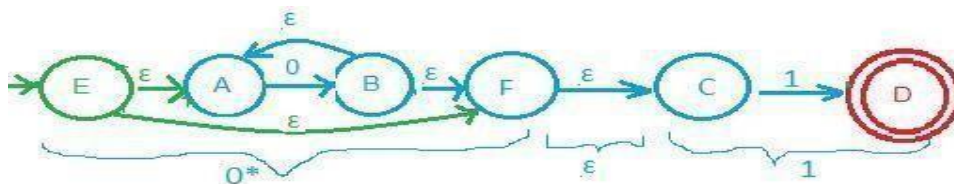


Figure 2.12: Finite automata for regular expression 0^*1 .

Example 2.2: Design a finite automata for the regular expression $0(0+1)^*1$

- i. Draw the finite automata for the symbol '0' and '1'

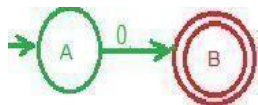


Figure 2.13: Finite automata for regular expression 0.

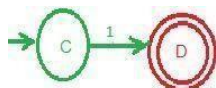


Figure 2.13: Finite automata for regular expression 1.

- ii. Construct the finite automata for the expression $0+1$

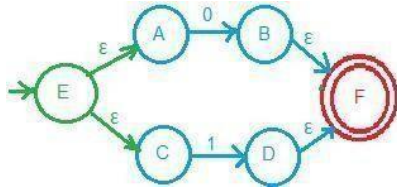


Figure 2.14: Finite automata for regular expression $0+1$.

- iii. Construct the finite automata for the symbol $(0+1)^*$

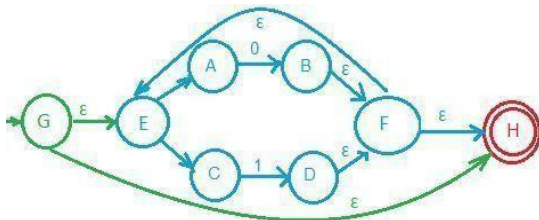


Figure 2.15: Finite automata for regular expression $(0+1)^*$.

- iv. Join the finite automata of 0 , $(0+1)^*$ and 1

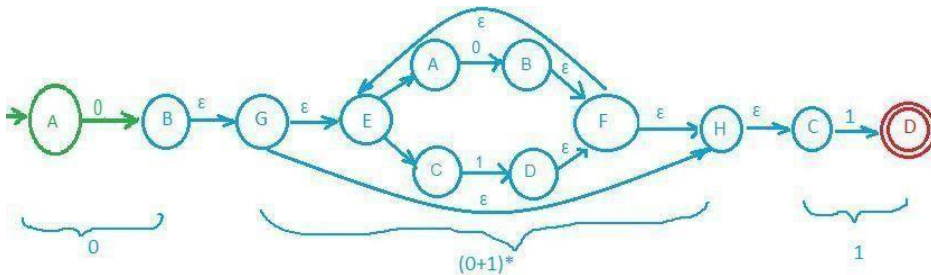


Figure 2.16: Finite automata for regular expression $0(0+1)^*1$.

Conversion from Finite Automata to Regular Expression:

They are of two methods of conversions, they are:

- i. Recursive Method.
- ii. State Elimination Method.

i. Recursive Method :

In this method it uses recursive method to construct new regular expression. For example the regular expression of DFA shown in figure 2.17 is

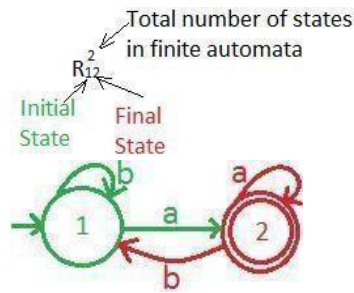


Figure 2.17: A typical finite automata.

In order to obtain the value of regular expression R_{12}^2 , we use all the (transition) possible regular expression of the DFA. The possible regular expression of the figure 2.17 is shown in the table 2.1.

Table 2.1: Recursive table

	k=0	k=1
R_{11}^k		
R_{12}^k		
R_{21}^k		
R_{22}^k		

Equations to obtain the regular expression of all the transitions are as follows:

$$\text{if } k=0 \quad R_{ij}^k = \begin{cases} a & \text{if } i \neq j \\ a \cup \epsilon & \text{if } i = j \end{cases}$$

where a is the transition between state i and j

$$\text{if } k > 0 \quad R_{ij}^k = R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1} + R_{ij}^{k-1}$$

If state 2 and 3 are final states then the regular expression is written as:

$$RE = R_{12}^3 + R_{13}^3$$

Example 2.3: Find the regular expression of DFA shown in figure 2.17 by using recursive method.

	k=0	k=1
R_{11}^k	$b + \epsilon$	b^*
R_{12}^k	a	$b^* a$
R_{21}^k	b	$b^* b$
R_{22}^k	$a + \epsilon$	$b^* a + \epsilon$

$$R_{12}^1 = R_{11}^0 (R_{11}^0)^* R_{12}^0 + R_{12}^0$$

$$= (b + \epsilon) (b + \epsilon)^* a + a$$

$$= [(b + \epsilon) (b + \epsilon)^* + \epsilon] a$$

$$= [(b + \epsilon)^+ + \epsilon] a$$

$$= b^* a$$

$$R_{21}^1 = R_{21}^0 (R_{11}^0)^* R_{11}^0 + R_{21}^0$$

$$= b (b + \epsilon)^* (b + \epsilon) + b$$

$$= [(b + \epsilon)^* (b + \epsilon) + \epsilon] b$$

$$= [(b + \epsilon)^+ + \epsilon] b$$

$$= (b + \epsilon)^+ b$$

$$= b^* b$$

$$R_{22}^1 = R_{21}^0 (R_{11}^0)^* R_{12}^0 + R_{22}^0$$

$$= b b^* a + a + \epsilon = b^* a + a + \epsilon$$

$$= b^+ + \epsilon) a + \epsilon$$

$$= b^* a + \epsilon$$

$$R_{12}^2 = R_{12}^1 (R_{22}^1)^* R_{22}^1 + R_{12}^1$$

$$RE = b^* a (b^* a + \epsilon)^* b^* a + \epsilon + b^* a$$

Example 2.4: Find the regular expression of DFA shown in figure 2.18 by using recursive method.

Figure 2.18: DFA

	k=0	k=1	k=2	
R_{11}^k	$\varphi+\varepsilon$	ε	$(11)^*$	$R_{11}^1 = R_{11}^0(R_{11}^0)^*R_{11}^0 + R_{11}^0$ $= \varepsilon(\varepsilon)^*\varepsilon + \varepsilon = \varepsilon$
R_{12}^k	1	1	$1(11)^*$	$R_{12}^1 = R_{11}^0(R_{11}^0)^*R_{12}^0 + R_{12}^0$ $= \varepsilon(\varepsilon)^*1 + 1 = 1$
R_{13}^k	0	0	1^*0	$R_{12}^1 = R_{11}^0(R_{11}^0)^*R_{12}^0 + R_{12}^0$ $= \varepsilon(\varepsilon)^*1 + 1 = 1$
R_{21}^k	1	1	$(11)^*1$	$R_{12}^1 = R_{11}^0(R_{11}^0)^*R_{12}^0 + R_{12}^0$ $= \varepsilon(\varepsilon)^*1 + 1 = 1$
R_{22}^k	ε	$11+\varepsilon$	$(11)^*$	$R_{13}^1 = R_{11}^0(R_{11}^0)^*R_{13}^0 + R_{13}^0$ $= \varepsilon(\varepsilon)^*0 + 0$ $= 0 + 0 = 0$
R_{23}^k	0	$10+0$	$(11)^*0(1+\varepsilon)$	
R_{31}^k	φ	φ	$11(11)^*$	
R_{32}^k	1	1	$1(11)^*$	
R_{33}^k	$0+\varepsilon$	$0+\varepsilon$	$101^*0+0+\varepsilon$	

$$\begin{array}{l}
 R_{31}^0 = R_{31}^0 (R_{11}^0) R_{11}^0 + R_{31}^0 \\
 = \Phi + \Phi = \Phi^* \\
 R_{32}^1 = R_{31}^0 (R_{11}^0) R_{12}^0 + R_{32}^0 \\
 = \Phi + 1 = 1 \\
 R_{33}^1 = R_{31}^0 (R_{11}^0) R_{13}^0 + R_{33}^0 \\
 = \Phi + 0 + \epsilon \\
 R_{11}^2 = R_{12}^1 (R_{22}^1) R_{21}^1 + R_{11}^1 \\
 1(11 + \epsilon) 1 + \epsilon \\
 = 11(11)^* + \epsilon \\
 = (11)^+ + \epsilon \\
 = (11)^* \\
 R_{12}^2 = R_{12}^1 (R_{22}^1) R_{22}^1 + R_{12}^1 \\
 = 1(11 + \epsilon)^* (11 + \epsilon) + 1 \\
 = 1(11 + \epsilon)^+ + 1 \\
 = 1[(11 + \epsilon)^+ + \epsilon] \\
 = 1(11)^*
 \end{array}
 \quad
 \begin{array}{l}
 R_{13}^1 = R_{12}^1 (R_{22}^1) R_{23}^1 + R_{13}^1 \\
 = 1(11 + \epsilon)^* (10 + 0) + 0 \\
 = 1(11)^* (1 + \epsilon) 0 + 0 \\
 = [11(11)^* + 1(11)^*] 0 + 0 \\
 = [(11)^+ + 1(11)^* + \epsilon] 0 \\
 = [(11)^* + 1(11)^*] 0 \\
 = 1^* \\
 R_{21}^1 = R_{22}^1 (R_{22}^1) R_{21}^1 + R_{21}^1 \\
 = (11 + \epsilon)(11 + \epsilon)^* 1 + 1 \\
 = (11 + \epsilon)^+ 1 + 1 \\
 = ((11 + \epsilon)^+ + \epsilon) 1 \\
 = (11)^* 1 \\
 R_{22}^2 = R_{22}^1 (R_{22}^1) R_{22}^1 + R_{22}^1 \\
 = (11 + \epsilon)(11 + \epsilon)^* (11 + \epsilon) + (11 + \epsilon) \\
 = ((11 + \epsilon)^+ + \epsilon) (11 + \epsilon) \\
 = (11 + \epsilon)^* (10 + 0) \\
 = (11)^* 0 (1 + \epsilon) \\
 R_{31}^2 = R_{32}^1 (R_{22}^1) R_{21}^1 + R_{31}^1 \\
 = 1(11 + \epsilon)^* 1 + \Phi \\
 = 11(11)^*
 \end{array}
 \quad
 \begin{array}{l}
 = (11 + \epsilon)(11 + \epsilon)^* (11 + \epsilon) + (11 + \epsilon) \\
 = ((11 + \epsilon)(11 + \epsilon)^+ + \epsilon) (11 + \epsilon) \\
 = ((11 + \epsilon)^+ + \epsilon) (11 + \epsilon) \\
 = ((11 + \epsilon)^+ + \epsilon) (11 + \epsilon) \\
 = (11)^* \\
 R_{23}^2 = R_{22}^1 (R_{22}^1) R_{23}^1 + R_{23}^1 \\
 = (11 + \epsilon)((11 + \epsilon)^* (10 + 0) + (10 + 0)) \\
 = [(11 + \epsilon)((11 + \epsilon)^* + \epsilon)] (10 + 0) \\
 = [(11 + \epsilon)^+ + \epsilon] (10 + 0) \\
 = (11 + \epsilon)^* (10 + 0) \\
 = (11)^* 0 (1 + \epsilon) \\
 R_{32}^2 = R_{32}^1 (R_{22}^1) R_{22}^1 + R_{32}^1 \\
 = 1(11 + \epsilon)^* (11 + \epsilon) + 1 \\
 = 1((11 + \epsilon)^+ + 1) \\
 = 1[(11 + \epsilon)^+ + \epsilon] \\
 = 1(11 + \epsilon)^* \\
 = 1(11)^* \\
 R_{33}^2 = R_{32}^1 (R_{22}^1) R_{23}^1 + R_{33}^1 \\
 = 1(11 + \epsilon)^* (10 + 0) + (0 + \epsilon) \\
 = 10(11 + \epsilon)^* (1 + \epsilon) + (0 + \epsilon) \\
 = 10(11)^* (1 + \epsilon) + (0 + \epsilon) \\
 = 101^* + 0 + \epsilon
 \end{array}$$

$$RE = R_{12}^3 + R_{13}^3$$

$$= [1^* 00(11^*)^*] 1(11)^* + 1(11)^* (101^* + 0 + \varepsilon)^* (101^* + 0 + \varepsilon) + 1^* 0$$

StateEliminationMethod:

In state elimination method, we obtain regular expression, by eliminating all the intermediate states. After eliminating the intermediate states, it is easy to find the regular expression for the remaining states (initial and final).



Figure2.19:Finiteautomata

The regular expression of the finite automata shown in figure 2.20 is:

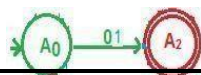


Figure 2.20: Finite automata

RE=01

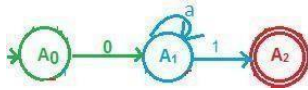


Figure 2.21: Finite automata

The regular expression of the finite automata shown in figure 2.2 is:

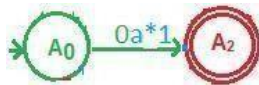


Figure 2.22: Finite automata

RE=0a*1

From the above examples we come to know that eliminating state has one predecessor state A_0 and one successor state A_2 . In complex finite automata the eliminating state may have more number of predecessors and successors. One such example is shown in the figure 2.23.

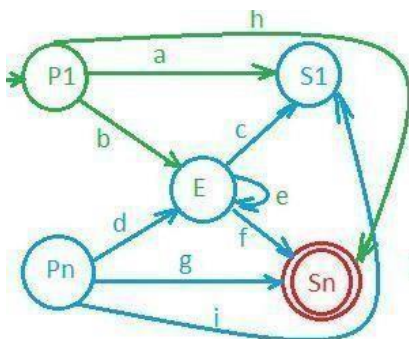


Figure 2.23: Eliminating state E has n predecessor and n successor.

Before eliminating any state from the finite automata, we need to find out the path passing through the eliminating state. The paths passing through the eliminating state are $P1ESn$, $P1ES1$, $PnES1$ and $PnESn$. After eliminating state E, the automata is shown in the figure 2.24.

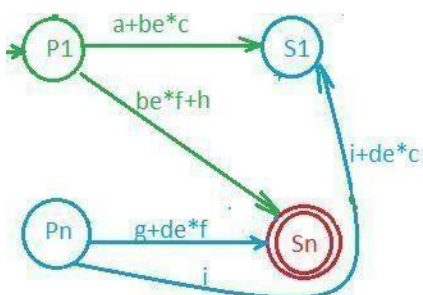


Figure 2.24: Finite automata after eliminating state E.

After eliminating all predecessors and successors, leaving initial state and final state, the remaining machine looks shown in the figure 2.25.

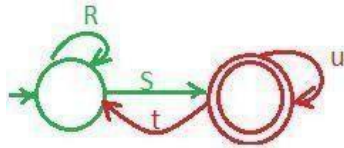


Figure 2.25: Finite automata after eliminating all predecessors and successors.

The regular expression is, language accepted by the initial state of finite automata and Su^* .

The language accepted by the initial state is $\{R^* + Su^*t + R^*Su^*t + su^*tR^* \dots\}$, this is like $\{x^* + y^* + xy + yx \dots\}$

We can simplify it as $(x+y)^*$ similarly, we can simplify the language for the initial state as $(R+Su^*t)^*$. Therefore, regular expression of finite automata is $(R+Su^*t)^*Su^*$. Note that in some cases the same state is initial as well as final state.



Figure 2.26: Same state is initial as well as final state. The

regular expression is R^* .

To eliminate state, three rules are used, they are:

I rule:

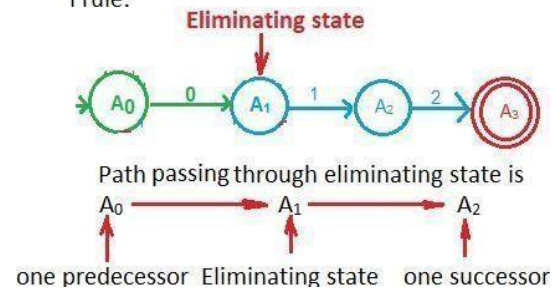


Figure 2.27: One path passing through eliminating state A_1

II rule:

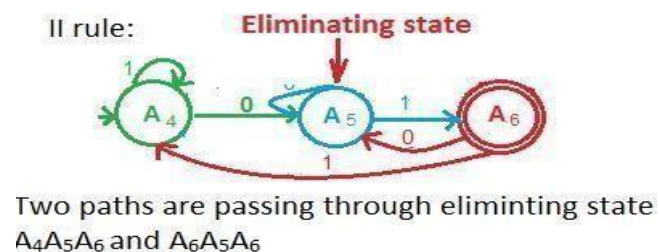
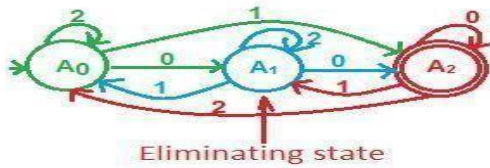


Figure 2.28: Two paths are passing through eliminating state A_1

Predecessor state	Eliminating State	Successor state
A_4	A_5	A_6
A_6	A_5	A_6

Rule III



Four paths are passing through eliminating state
 $A_0A_1A_2$, $A_0A_1A_0$, $A_2A_1A_0$ and $A_2A_1A_2$

Figure 2.29: Four paths are passing through eliminating state A_1

Predecessor state	Eliminating State	Successor state
A_0	A_1	A_2
A_0	A_1	A_0
A_2	A_1	A_0
A_2	A_1	A_2

Example 2.5: Construct regular expression for the finite automata shown in figure 2.30.

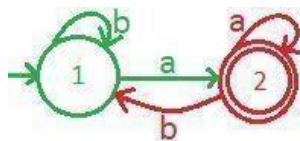


Figure 2.30: Finite Automata

No intermediate in the given finite automata, hence we can't eliminate any of the state.

$R=b$; $S=a$; $u=a$; $t=b$;

Therefore $RE=(R+Su^*t)^*Su^*$

$= (b+aa^*b)^*aa^*$

$= b(\epsilon+aa^*)^*aa^*$

$= b(aa^*)^*aa^*$

Example 2.6: Construct regular expression for the finite automata shown in figure 2.31 by using state elimination method.

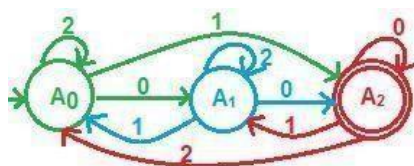


Figure 2.31: Finite Automata.

We can eliminate one intermediate state A_1 , the paths passing through the eliminating state are $A_0A_1A_0$, $A_0A_1A_2$, $A_2A_1A_0$ and $A_2A_1A_2$.

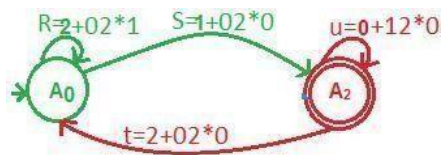


Figure 2.32: Final finite automata

$$RE = (R + Su^*t)^* Su^*$$

$$= ((2+02^*1) + (1+02^*0)(0+12^*0)^*(2+02^*0))^* ((1+02^*0)(0+12^*0)^*).$$

Application of Regular expression:

Pumping lemma:

It is used to prove a language as non-regular for example

$$L = \{00, 000, 0000, \dots\}$$

The automaton is shown in figure 2.33 which accepts 00.

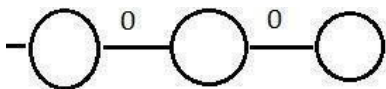


Figure 2.33: Finite automata

The automaton is shown in figure 2.34 which accepts 000.

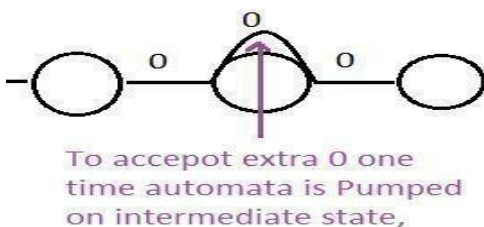


Figure 2.34: Finite automata pumped at intermediate state one time. The

automaton is shown in figure 2.35 accepts 0000.

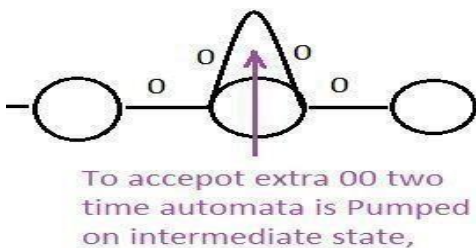


Figure 2.35: Finite automata pumped at intermediate state one time.

If more 0s, there will be more number of pumping. In such a case it is very difficult to count the number of pumping. To count the number of pump easily we draw circles around the intermediate state so that we can easily count the number of pumping to prove the language as non-regular.

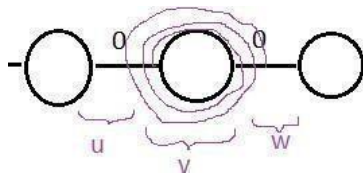


Figure 2.35: Finite Automaton pumped at intermediate state finitely. We

can write $uv^i w \in L$ for $i = \{1, 2, 3, \dots\}$.

To prove the language as non-regular, smallest string of language is divided into uvw by considering two conditions.

- i. $\forall \epsilon \in \Sigma$.
- ii. $|UV| \leq n$ where n is the smallest string of the language. To prove the language as non-regular, throughout the proof the value of n is constant.

Example 2.7: Prove that the language $L = \{0^i : i \geq 1\}$ is not regular.

$$L = \{0, \underbrace{0000}_n, \underbrace{0000000000}_{uv}, \dots\}$$

$$uv^i w \in L \text{ for } i = \{1, 2, \dots\}$$

$$0(00)^1 0000000 \in L$$

$$L = 10000000000$$

$$\in L \quad i = 2$$

$$00000000000 \notin L$$

When $i = 2$ the string obtained is not belong to language, hence proved language is not regular.

Example 2.8: Prove that the language $L = \{a^n b^{n+1} : n \geq 2\}$ is not regular.

$$L = \{\underbrace{aabb}_{n}, \underbrace{aaab}_{u} \underbrace{bbb}_{v}, \dots\}$$

$$i = 1$$

$$aaabbbb \in L \quad i = 2$$

$$Aaabaabbbb \notin L$$

When $i = 2$ the string obtained is not belong to language, hence proved the language is non-regular.

Closure Properties of Regular Expression:

Till now we have seen many regular expressions and their applications. These regular expressions are bounded/ related by operators called closure properties of regular expression. The different operators used to relate regular expressions are:

- i. Boolean.
- ii. Reversal.
- iii. String Homomorphism and Inverse Homomorphism.

i. Boolean Operator:

They are of different types

- a. Union
- b. Intersection
- c. Difference
- d. Complement.

a. Union Operation.

To perform union operation it uses following steps.

- This operation is applied on two machines defined as

$$M_1 = (Q_1, \Sigma_1, \delta_1, IS_1, FS_1)$$

$$M_2 = (Q_2, \Sigma_2, \delta_2, IS_2, FS_2)$$

- Find the language L_1 and L_2 of the two machines.
- Perform union operation on the two languages L_1 and L_2 to obtain language L_3
- Combine two machines to perform union operation.
- Apply language L_3 on combined machines to obtain machine for union operation.

Same steps are used for the other Boolean operation.

$$M_1 \cup M_2 = (Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, \delta_1 \cup \delta_2, ?, ?)$$

For example consider the machine 1 is shown in figure 2.36

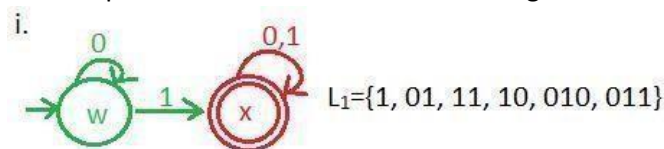


Figure 2.36: Finite Automata

Machine 2 is shown in figure 2.37

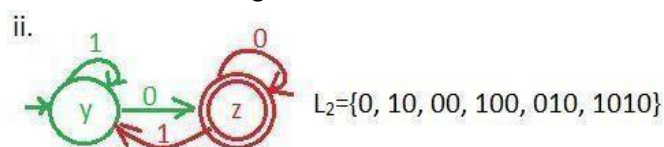


Figure 2.37: Finite Automata

The union operation on two languages is

$$L_1 \cup L_2 = \{0, 1, 00, 01, 10, 11, 010, 011, 010, 100, 1010\}$$

In order to obtain the finite automata for the union operation, we need to join two finite automata. The transition table for union operation is shown in table...

Table 2.1: Transition table for Boolean operation

$\delta_{1 \cup 2}$	0	1
w, y	w, z	x, y
w, z	w, z	x, y
x, y	x, z	x, y
x, z	x, z	x, y

Equivalent diagram is shown in the figure 2.38

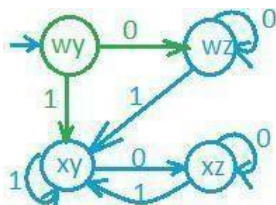


Figure 2.38: Finite Automata after combining figure 2.36 and 2.37.

According to union operation it has to accept 0, 00

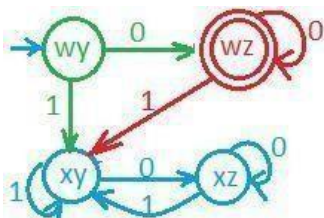


Figure 2.39: Finite Automata to accept 0, 00.

it has to accept 1, 01, 11, 011

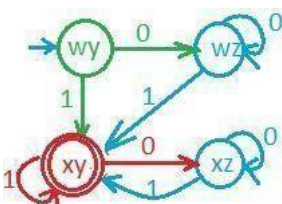


Figure 2.40: Finite Automata accepts 1, 01, 11, 011. It

also accepts 10, 010...

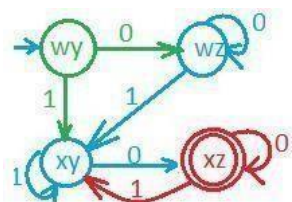


Figure 2.41: Finite Automata accepts 10, 010...

The final automata for the union operation is shown in figure 2.42

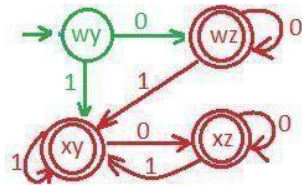


Figure 2.42: Union of two finite automata of figure 2.36 and 2.37. For intersection $M_1 \cap M_2 = \{10, 100, 010, 1010, \dots\}$

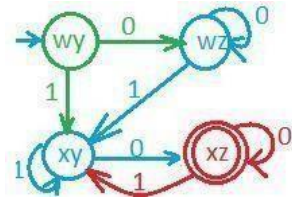
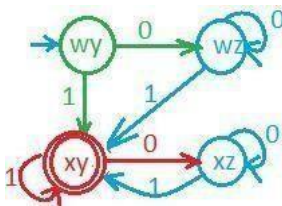


Figure 2.43: Intersection of two finite automata of figure 2.36 and 2.37. For



differentiation $M_{1-2} = \{1, 11, 01, 011, 101\}$

Figure 2.44: Difference of two finite automata of figure 2.36 and 2.37.

Compliment:

To compliment the regular expression it uses the following steps:

- Obtain the ϵ -NFA/NFA for the regular expression.
- Convert the ϵ -NFA to DFA.
- Compliment DFA.
- Convert DFA to regular expression.

Example 2.9: Find the compliment of regular expression 0.

- Obtain the ϵ -NFA/NFA for the regular expression.

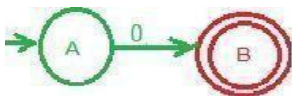


Figure 2.45: finite automata for regular expression 0.

- Convert the ϵ -NFA/NFA to DFA.

$$\delta(A, 0) = B; \delta(B, 0) = \varnothing; \delta(\varnothing, 0) = \varnothing$$

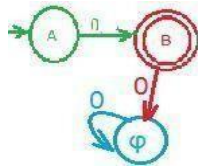


Figure 2.46: Equivalent DFA for the figure 2.45.

- iii. Compliment DFA. Reverse all transitions and convert all final to non final and vice-versa.

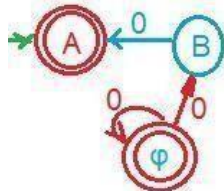


Figure 2.47: Complimented DFA of figure 2.46.

State B and ϕ are not reachable from the initial State. Hence the two states are removed from the finite automata.



Figure 2.48. Complimented DFA for the regular expression 0.

The regular expression of finite automata shown in figure 2.48 is

$$RE = \epsilon$$

Reversal of regular expression:

- Reverse all transitions.
- Convert initial state to final and final state to intermediate state.
- Draw epsilon-transition between new initial state to all previous final states, mark new state as an initial state.

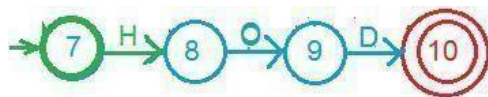


Figure 2.49: Non-deterministic finite automata.



Figure 2.50: Reversed NFA of the figure 2.49

String Homomorphism:

String homomorphism is a function on the language, it replaces string by symbol

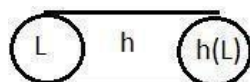


Figure2.51:Homomorphism.

InverseHomomorphism:

Itisaninversestringhomomorphism

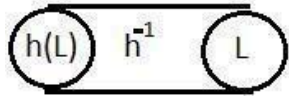


Figure2.52:Inversestringhomomorphism

Example2.10:Considerthehomomorphism, $h(0)=abb;h(1)=a;h(011)=abbaa$

If the homomorphism from an alphabet $\{0, 1, 2\}$ to $\{a, b\}$ is defined by $h(0)=ab$; $h(1)=b$; & $h(2)=aa$,then:

- i. Whatish(2201).
 - ii. IfLislanguage 1^*02^* whatish(L).
 - iii. IfLislanguage $(ab+baa)^*bab$ thenwhatish $^{-1}(L)$
- i. $h(2201)=aaaaabb$
 - ii. $h(L)=h(1^*02^*)=b^*ab(aa)^*$
 - iii. $(ab+baa)^*bab=(0+12)^*10$

Equivalenceandminimization:

In Equivalence, two different DFA's are given, we need to find out the two DFA's are similar or not. By filling table and Preshapat tree we can decide the two DFAs are similar | not similar.

Preshapat Tree: To find pair of nodes similar | not similar Preshapat tree is used. It consists of root node, intermediate nodes and leaves nodes. The root node of the tree is pair of initial states of two DFAs. Next we construct branches of tree depending on number of input symbols. If leaves node do not have a cross in the difference table and same pairs are repeating, we stop constructing tree and decide theroot pair is similar or else dissimilar, anyof leaves node hasa cross inthe differencetable.

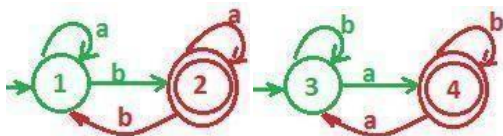


Figure2.53:Deterministicfiniteautomata.

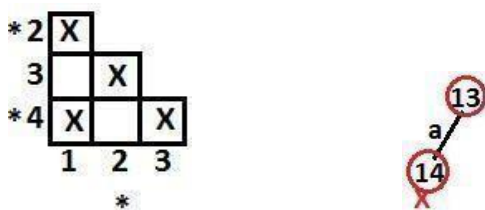


Figure2.54:DifferencetablewithPreshepat tree.

Leavesnode14hascrossinthedifferencetable,sotwoDFA'sarenotequal.

Forexample

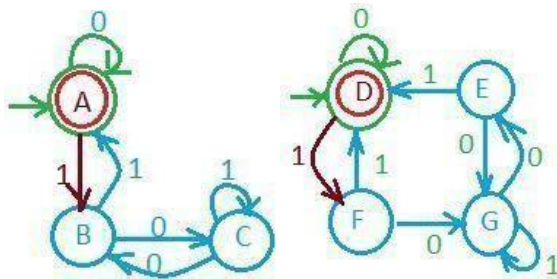


Figure2.55:Deterministicfiniteautomata.

Notethatrootofthetreeispaairofinitialstates.Leavesnodesofthetreedonothavecrossinthe difference table and nodes are repeating, hence the two DFA's are similar.

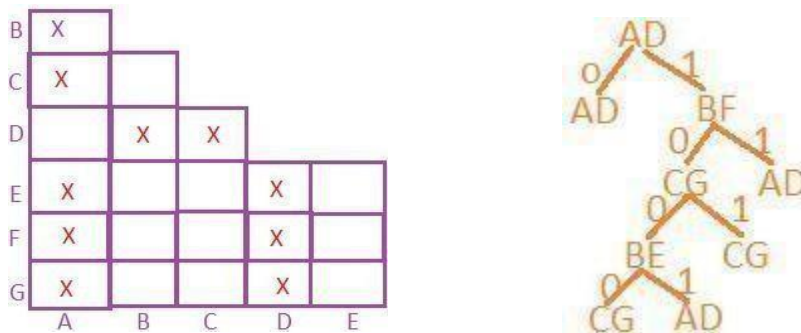


Figure2.56:DifferencetablewithPreshapattree.

Example2.11:FindoutthetwoDFA'saresimilarornot.

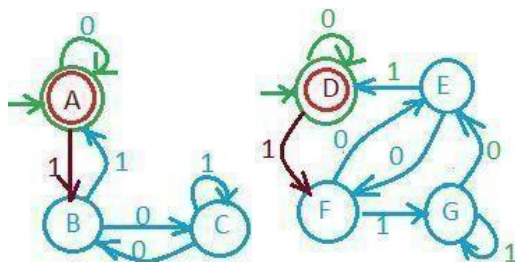


Figure2.57:Deterministicfiniteautomata.

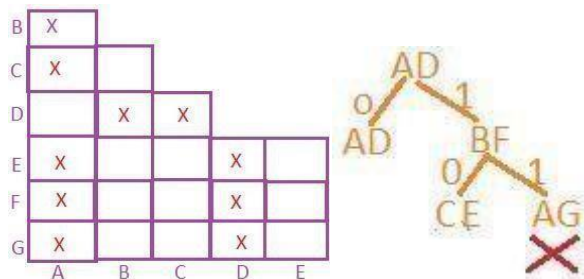


Figure 2.58: Transition tree with table.

The leaf node AG has X in the table; hence two DFAs are dissimilar.

In case of minimization, the two states are minimized to one state if they are similar. For example, state X and Y are equal, they are replaced by one state as shown in figure 2.59

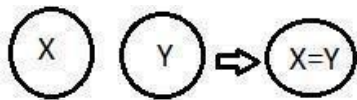


Figure 2.59: Similar states are replaced into a single state.

To decide if the states are similar or not, we use a Shanprepat tree.

Shanprepat Tree: To find equivalence between a pair of states, a Shanprepat tree is used. It consists of a root node, intermediate nodes, and leaf nodes. The root node of the tree is a pair of states used to find similar or not similar states. Next, we construct branches of the tree depending on the number of input symbols. If a leaf node does not have a cross in the difference table and some pairs are repeating, we stop constructing the tree and decide if the root pair is similar or else decide if two states are dissimilar, if any leaf node has a cross in the difference table.

Example 2.12: Using the table filling method, minimize the DFA shown in figure 2.60 and draw the transition diagram of the resulting DFA.

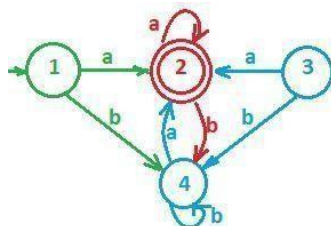


Figure 2.60: Deterministic finite automata.

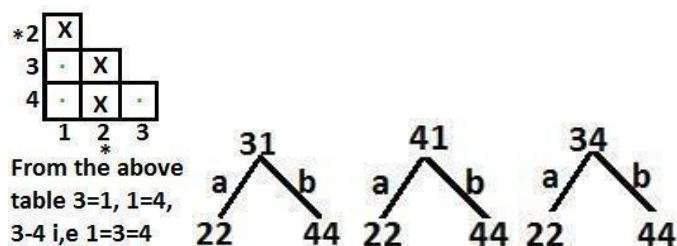


Figure 2.61: Difference table with Shanprepat Tree

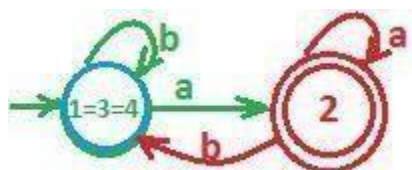


Figure 2.62: Minimized DFA of figure 2.60

Example 2.13: Using table filling method, minimize the DFA shown in figure 2.63 and draw the transition diagram of the resulting DFA.

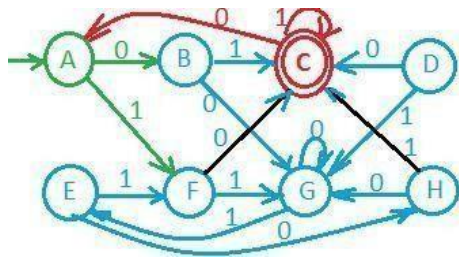


Figure 2.63: Deterministic finite automata.

From the given DFA we draw the dissimilarity table as shown in figure. As compared to final state C all states are different (non-final states), hence X is marked.

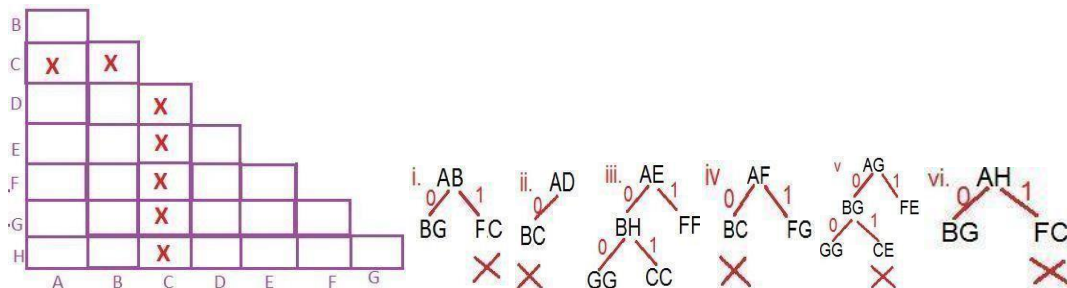


Figure 2.64: Difference table with Shanprepat Tree.

The leaf node decides the root node. Any leaf node has X in the table, the root node is marked as X or else pair node repeats. Stop when the pair nodes start repeating, and decide the root node pair is similar.

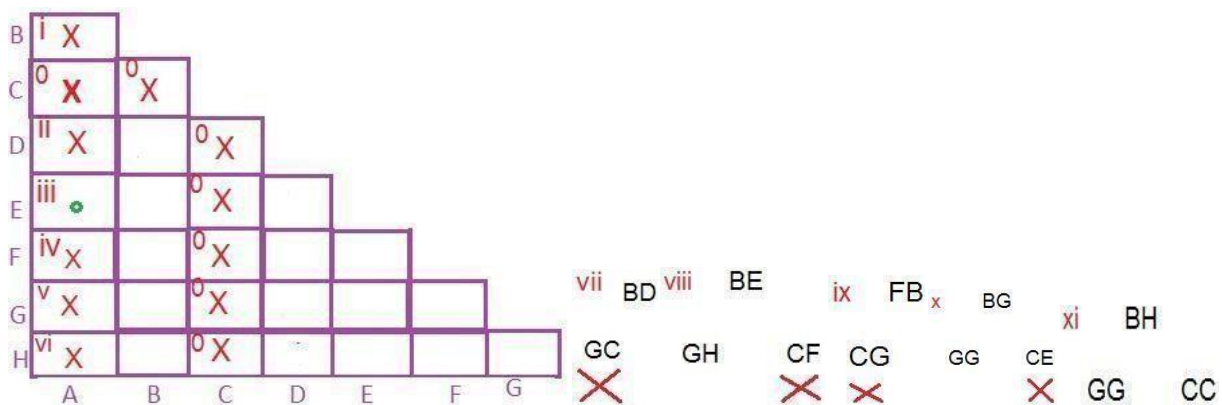


Figure 2.64: Difference table with Shanprepat Tree.

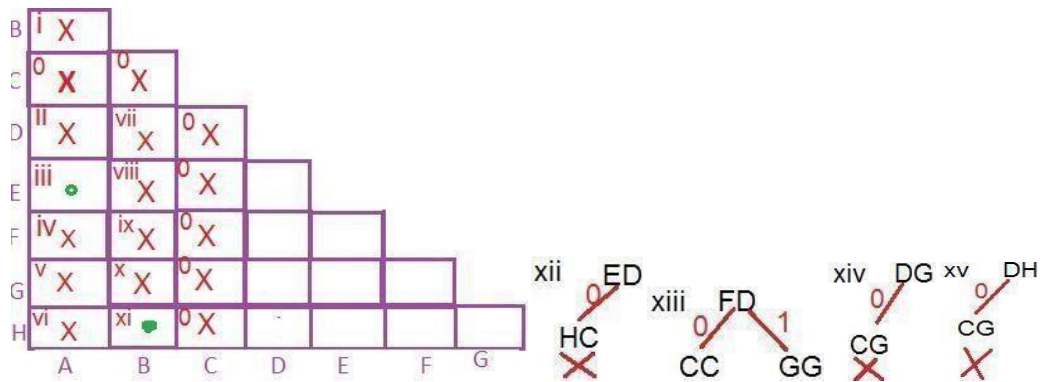


Figure 2.65: Difference table with Shanprepat Tree.

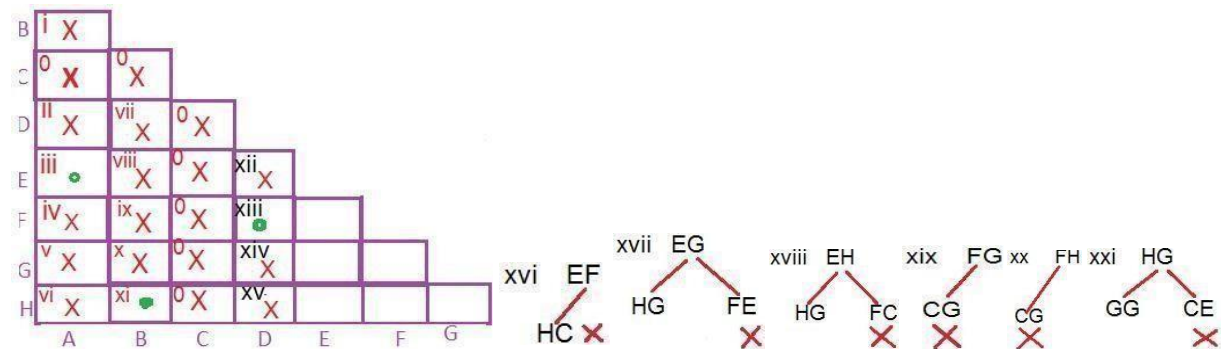


Figure 2.66: Difference table with Shanprepat Tree.

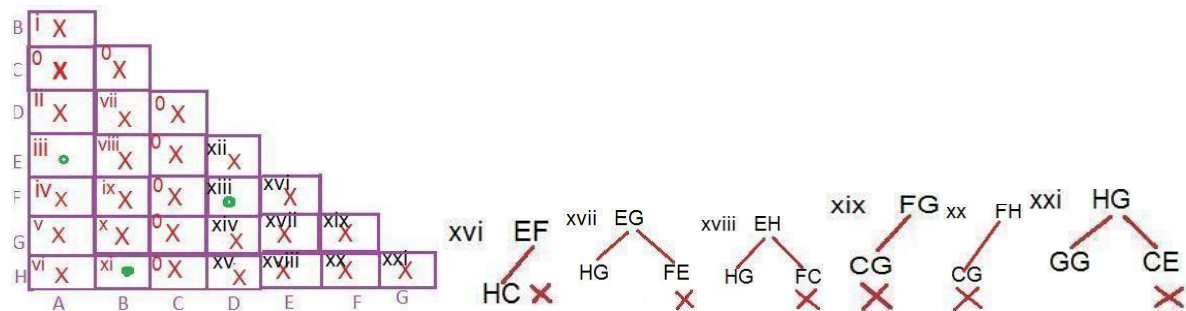


Figure 2.66: Difference table with Shanprepat Tree.

From the table, we come to know that $A=E$, $B=H$ and $D=F$. the resulting DFA is shown in figure..

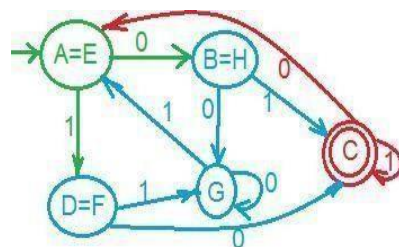


Figure 2.67: Minimized DFA of the figure 2.63

Example 2.14: Using table filling method, minimize the DFA shown in table and draw the transition diagram of the resulting DFA.

δ	0	1
$\rightarrow A$	B	E
B	C	F
*C	D	H
D	E	H
E	F	I
*F	G	B
G	H	B
H	I	C
*I	A	E

The table shows the dissimilarity in the states of the given problem.

B								
C	X	X						
D			X					
E			X					
F	X	X		X	X			
G			X			X		
H			X			X		
I	X	X		X	X		X	X
	A	B	C	D	E	F	G	H

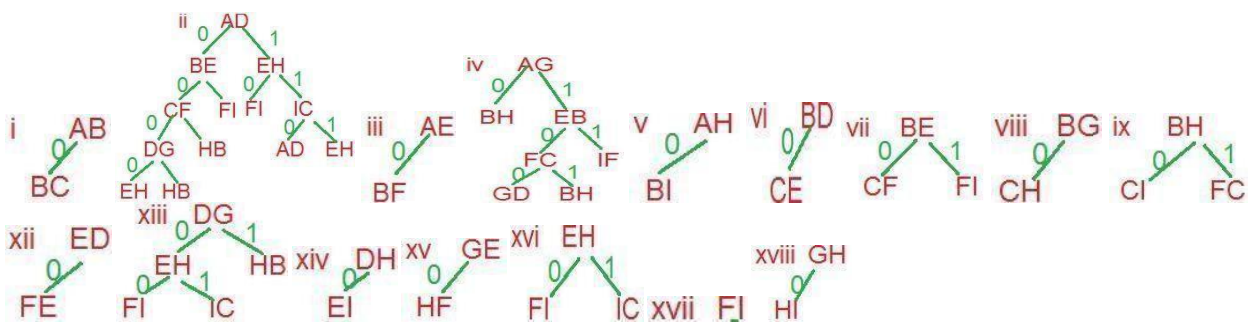


Figure 2.68: Difference table with Shanprepat Tree.

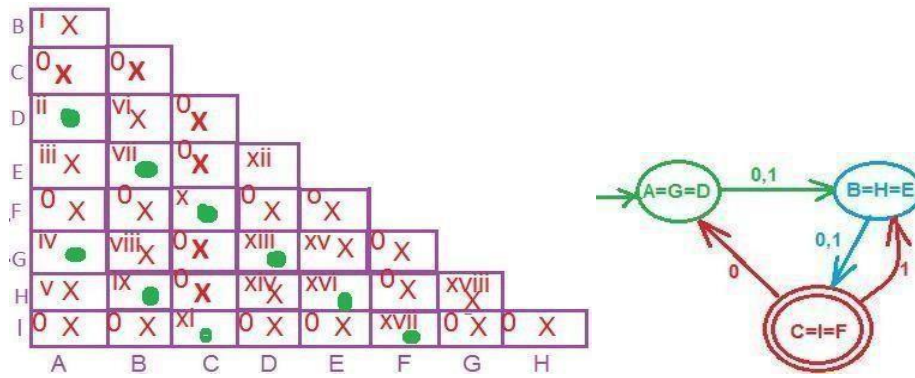


Figure 2.69: Difference table with minimized DFA.